Classes, Objects and the Four Pillars

Object-Oriented Programming - Unit 1 - Core OOP concepts

Procedural vs object-oriented

Procedural code organizes a program around functions that act on shared data. Object-oriented programming (OOP) bundles data and the operations on it into objects, which makes large programs easier to reason about, reuse and test.

Classes and objects

A class is a blueprint that defines fields (state) and methods (behaviour). An object is an instance of a class with its own state. Constructors initialize new objects.

```
class Account {
    private double balance;           // encapsulated state
    public Account(double opening) { balance = opening; }
    public void deposit(double amt) {
        if (amt <= 0) throw new IllegalArgumentException();
        balance += amt;
    }
    public double getBalance() { return balance; }
}
```

The four pillars

- Encapsulation: hide internal state behind a public interface (private fields + methods) so invariants cannot be broken from outside.
- Abstraction: expose what an object does, not how - via abstract classes and interfaces.
- Inheritance: a subclass reuses and extends a superclass (an is-a relationship).
- Polymorphism: one interface, many behaviours - the method that runs depends on the actual object type.

Static vs instance members

A static member belongs to the class itself and is shared by all objects; an instance member belongs to a single object. Static methods cannot use this.

> *Exam tip: constructor overloading is compile-time polymorphism; constructors are never inherited, but a subclass constructor calls super(...) first.*